

Курсов проект по Обектно-ориентирано програмиране 1

Проект 9: Склад
Дарин Росенов Пенчев
СИТ, Група 1
24621653
Май 2026

Задание на курсовия проект

Да се напише програма, реализираща информационна система, обслужваща склад. Програмата съхранява и обработва данните за наличността в склада във файл. За всеки продукт се съхранява следната информация:

- име (описание на продукта, символен низ с произволна дължина)
- срок на годност
- дата на постъпване в склада
- име на производител
- мерна единица (килограми, литри)
- налично количество
- местоположение (секция/рафт/номер)
- номерирайте склада си, както прецените, че ще ви е удобно, имайте предвид, че в началото той е празен и различно количество стока е нормално да заема различно по обем място
- коментар (свободен текст)

След като приложението отвори даден файл, то трябва да може да извършва посочените по-долу операции, в допълнение на общите операции (open, close, save, save as, help u exit):

print	Извежда информация за наличните продукти в склада
add	Добавя нов продукт в склада, като въвежда всички данни за продукта и ги проверява за коректност. При добавянето на продукта се спазват следните правила: • ако нов продукт е с различен срок на годност от вече съществуващ едноименен продукт, той да бъде поставен на различно място • ако имате достатъчно място, еднакви продукти с един и същи срок на годност да бъдат поставени на едно и също място • при извеждане на списъка с налични продукти да се изведе общото количество на едноименните продукти, независимо от срока им на годност
remove	Изважда продукт от склада: • по дадено име и количество изважда съответните продукти от склада и извежда информацията за продукта и къде се е намирал е • при наличие на повече от една партида, прво намалява тази със най-скоро изтичащ срок на годност, тогава в информацията за извършеното действие се отбелязва количеството и мястото на всяка от партидите, които сме намалили

	в случай на опит за изваждане на повече от наличното да се дава информация на потребителя за наличността на продукта и срока на годност на партидите и възможност да реши дали все пак не иска да извади това което е останало
log < from> <to>	Извежда справка за всички промени в наличността в периода от дата <from> до дата <to>, включително зареждания и извеждания на стоки.
clean	Разчиства склада от всички стоки, на които е изтекъл или предстои скоро да изтече срока на годност, като извежда информация за разчистените стоки

Съдържание

Задание на курсовия проект	1
Съдържание	1
Глава 1. Увод	2
1.1 Описание и идея на проекта	2
1.2 Цел и задачи на разработката	2
Глава 2. Преглед на предметната област	3
2.1 Основни концепции и алгоритми	3
2.2 Подходи и методи за решаване	3
2.3 Функционални и нефункционални изисквания	4
Глава 3. Проектиране	5
3.1 Обща структура на проекта	5
3.2 Диаграми / блок схеми	8
Глава 4. Реализация	14
4.1 Реализация на класове	14
4.2 Алгоритми и оптимизации	15
Глава 5. Тестване	17
Глава 6. Заключение	18

Връзка към хранилището на проекта: <https://github.com/darin2302/oop-project>.

Глава 1. Увод

1.1 Описание и идея на проекта

Проектът реализира информационна система за управление на склад. Системата приема входни XML файлове, описващи текущата наличност и историята от действия, и предлага на потребителя интерактивен команден интерфейс, чрез който той може да добавя, изважда, разчиства и анализира продукти. Целта на разработката е не само да се реши изброеният в заданието набор от команди, но и да се демонстрира приложението на основните принципи и шаблони на обектно-ориентираното програмиране в реална, макар и опростена, бизнес задача.

Системата следва модела *load-edit-save*: файлът се чете еднократно при *open*, веднага след това се затваря, всички промени се пазят в паметта и се пишат обратно само при изрично подадена команда *save* или *save as*.

1.2 Цел и задачи на разработката

Основната цел е да се изгради приложение, което чете и записва валидни XML файлове, поддържа всички задължителни и допълнителни команди от заданието и налага коректно бизнес правилата за добавяне (сливане на еднакви партиди), изваждане (FIFO по срок на годност) и разчистване (по срок).

Допълнителни задачи, които си поставихме:

- Прилагане на минимум шест шаблона за проектиране за четим, разширим и тестваем код.
- Покриване на крайни случаи (празен склад, недостатъчно количество, разваляне на входния файл).
- Изграждане на стабилна архитектура, в която нова команда се добавя без промяна на съществуващ код (Open-Closed принцип).

Глава 2. Преглед на предметната област

2.1 Основни концепции и алгоритми

Партида (Batch). Партидата е минималната единица, която складът пази. Тя обединява име на продукт, производител, мерна единица, количество, срок на годност, дата на постъпване, местоположение и коментар. Две партии с едно и също име, но различен срок на годност, са различни единици и заемат различни места. При еднакво име и еднакъв срок на годност, новопостъпило количество се влива в съществуващата партида.

Местоположение (Location). Складът е разделен на секции (A–E), всяка секция има пет рафта, всеки рафт има десет места — общо 250 свободни позиции. Заданието изрично разрешава „номерирайте склада си както прецените“; избран е модел *една позиция = една партида*.

Журнал (LogEntry). Всяко добавяне и изваждане се записва в журнал с момент (до секунда), тип на действието (ADD или REMOVE), име на продукта, количество и местоположение. Журналът е append-only.

Алгоритъм за FIFO изваждане. Партидите на даден продукт се сортират по срок на годност възходящо; количеството се източва от най-скоро изтичащата партида. Ако тя не достига, остатъкът се взима от следващата по срок и т.н., докато се удовлетвори искането или се изчерпи наличността.

Алгоритъм за разчистване. При команда `clean` се намират всички партии с `expiryDate <= today + 3 days`. Тези партии се премахват от склада, всяко премахване се записва в журнала, след което се изчислява финансовата загуба по въведена от потребителя цена.

2.2 Подходи и методи за решаване

Решението стъпва на следните методи и стандарти:

- **JAXB** (Jakarta XML Binding) — за маршалинг между Java обекти и XML, тъй като заданието фиксира XML като формат за съхранение.
- **java.time** — стандартното API на Java SE 17 за работа с дати и периоди (`LocalDate`, `LocalDateTime`).
- **Обектно-ориентирани шаблони за проектиране** — Command, Factory, Builder, Singleton, Strategy и Observer, разяснени в Глава 3.

- **Принципите SOLID** — особено Single Responsibility (всеки клас има една причина за промяна) и Open-Closed (нови команди и стратегии се добавят, без да се променя съществуващ код).
- **По-конкретно - Dependency Inversion чрез интерфейс ConsoleIO** — командите не зависят от System.in/System.out, а от абстракция, която при нужда може да бъде заменена.

2.3 Функционални и нефункционални изисквания

Функционални изисквания. Системата трябва да поддържа командите от Таблица 1.

Таблица 1. Функционални команди и изискванията към тях.

#	Команда	Изискване от заданието
1	open <file>	Зарежда XML или създава нов празен; затваря файла след прочитането
2	close	Изчиства паметта, забранява всички операции освен open/help/exit
3	save	Записва обратно в текущия път
4	save as <file>	Записва на нов път и го прави текущ
5	print	Извежда наличните партии и общото количество на едноименните продукти
6	add	Интерактивно добавя нов продукт; слива при съвпадение име+срок
7	remove <name> <qty>	FIFO по срок; при недостиг — потвърждение от потребителя
8	log <from> <to>	Извежда журналните записи в дадения период (включително двата края)
9	clean	Премахва изтекли и скоро изтичащи (до 3 дни) партии; пита за цена и пресмята загуба
10	loss <p> <price> <from> <to>	Историческа справка за загубата на даден продукт в зададен период
11	help, exit	Стандартни помощни команди

Нефункционални изисквания.

- **Коректност на формата.** Записаните файлове трябва да бъдат валиден XML, който системата може да зареди повторно без загуба.
- **Устойчивост на грешки.** Невалиден потребителски вход не трябва да поваля програмата — REPL цикълът продължава да работи. Изключение: повреден входен файл при open — заданието изисква програмата да прекрати изпълнението си.
- **Разширяемост.** Нови команди и стратегии за изваждане се добавят с минимум промени в съществуващия код.
- **Прозрачност.** Всяко действие, което променя състоянието, се записва в журнала.

Глава 3. Проектиране

3.1 Обща структура на проекта

Кодът е разделен на единадесет пакета, всеки със собствена отговорност (Таблица 2).

Таблица 2. Пакети и техните отговорности.

Пакет	Отговорност
<code>bg.warehouse.cli</code>	REPL цикъл — чете вход, изпраща към команда, обработва централизирани изключения
<code>bg.warehouse.command</code>	Интерфейс <code>Command</code> , <code>CommandFactory</code> , тук се регистрират всички конкретни команди
<code>bg.warehouse.command.impl</code>	Конкретните команди (<code>OpenCommand</code> , <code>AddCommand</code> и т.н.)
<code>bg.warehouse.model</code>	Доменни класове и енумерации — <code>Warehouse</code> , <code>Batch</code> , <code>Product</code> , <code>Location</code> , <code>LogEntry</code> , <code>Unit</code> , <code>LogAction</code>
<code>bg.warehouse.service</code>	Бизнес слой — <code>WarehouseService</code> (<code>Receiver</code>), <code>LocationAllocator</code> , стратегиите за изваждане
<code>bg.warehouse.session</code>	<code>WarehouseSession</code> (<code>Singleton</code>) — единственото място, което знае текущия отворен файл

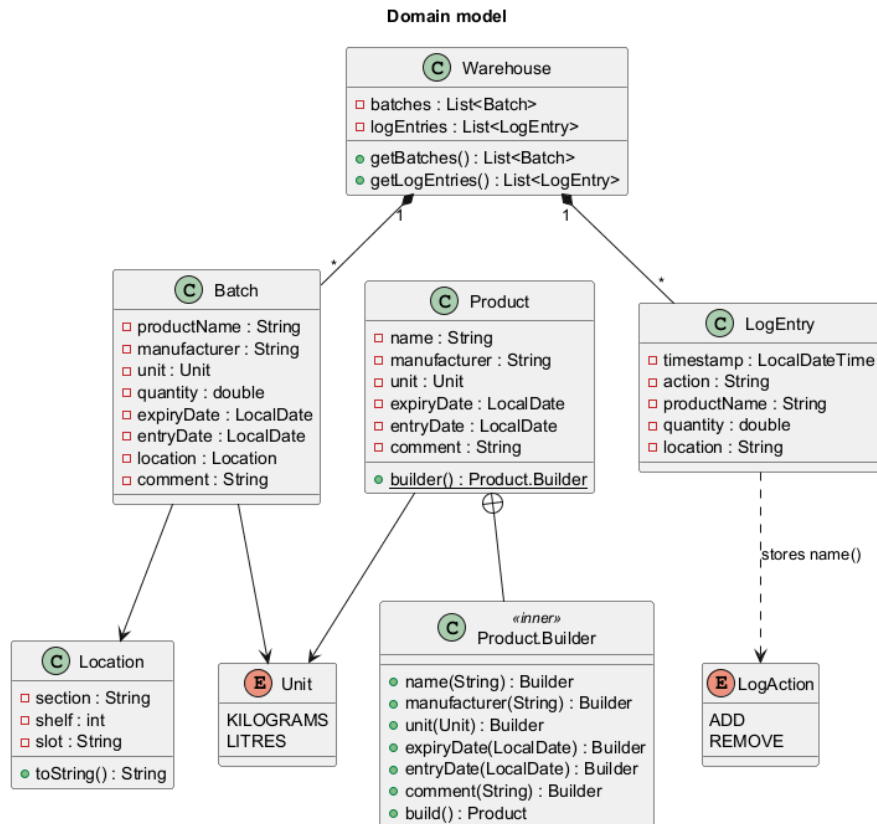
Пакет	Отговорност
<code>bg.warehouse.observer</code>	Интерфейс <code>WarehouseEventListener</code> и <code>AuditLogger</code> — Observer шаблон
<code>bg.warehouse.exception</code>	Собствените изключения, прехвърляни от <code>Receiver</code> и улавяни в CLI
<code>bg.warehouse.io</code>	<code>ConsoleIO</code> интерфейс и <code>SystemConsoleIO</code> — абстракция над <code>System.in/System.out</code>
<code>bg.warehouse.xml</code>	JAXB маршалинг и адаптери за <code>LocalDate</code> / <code>LocalDateTime</code>
<code>bg.warehouse.util</code>	Споделени константи и помощни функции

Шаблоните за проектиране, заложи в архитектурата, са обобщени в Таблица 3.

Таблица 3. Приложени шаблони за проектиране.

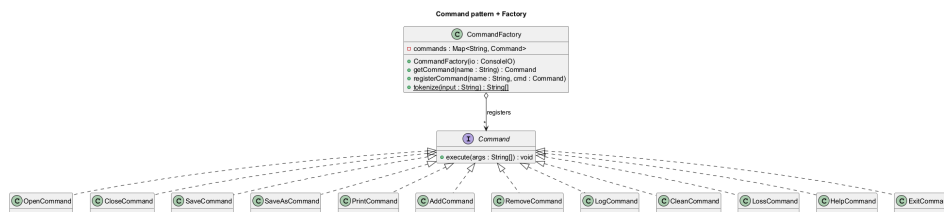
Шаблон	Място на приложение	Цел
Command	Command интерфейс, всяка REPL команда е реализация	Изолира всяка команда; нова команда се добавя без промяна в съществувашите
Factory	CommandFactory	Регистрира команди в <code>HashMap<String, Command></code>

Шаблон Builder	Място на приложение Product.Builder	Цел Констру ира Product с дълъг и валидиру ем списък от полета
Singleton	WarehouseSession	Единстве н носител на „текущ файл и неговото съдържа ние“
Strategy	RemovalStrategy + ExpiryFirstRemovalStrategy	Алгорит ъмът за избор на партиди при изважда не е сменяем
Observer	WarehouseEventListener + AuditLogger	Warehous eService уведомяв а за всяко добавяне /изважда



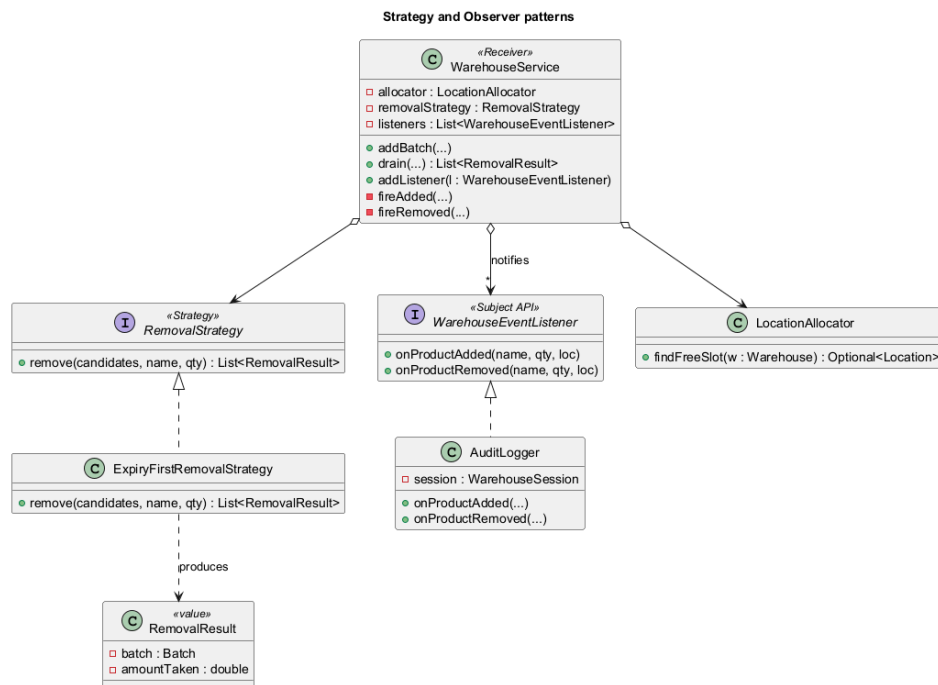
Фигура 2. Доменен модел.

Командният шаблон и регистрацията през фабрика са показани на Фигура 3.



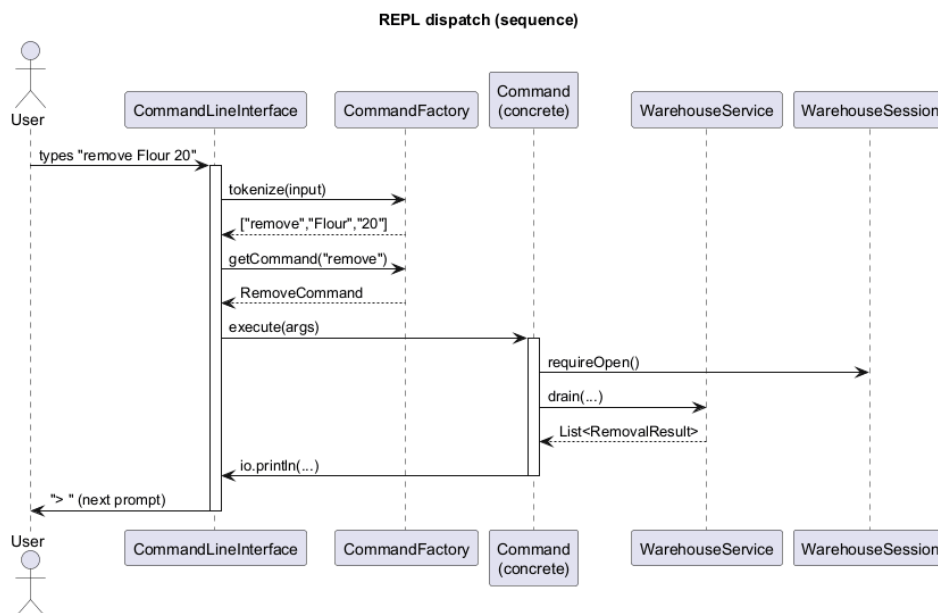
Фигура 3. Command + Factory.

Стратегията и наблюдателят разтоварват WarehouseService от две независими отговорности — как се изважда и как се записва (Фигура 4).



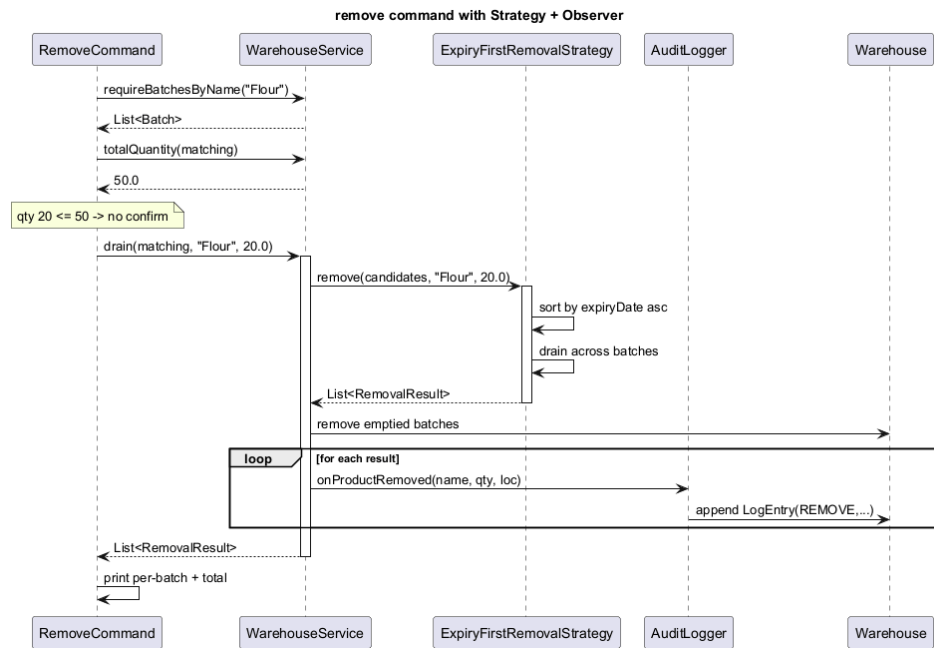
Фигура 4. Strategy + Observer.

Изпращане в REPL цикъла е показано на Фигура 5.



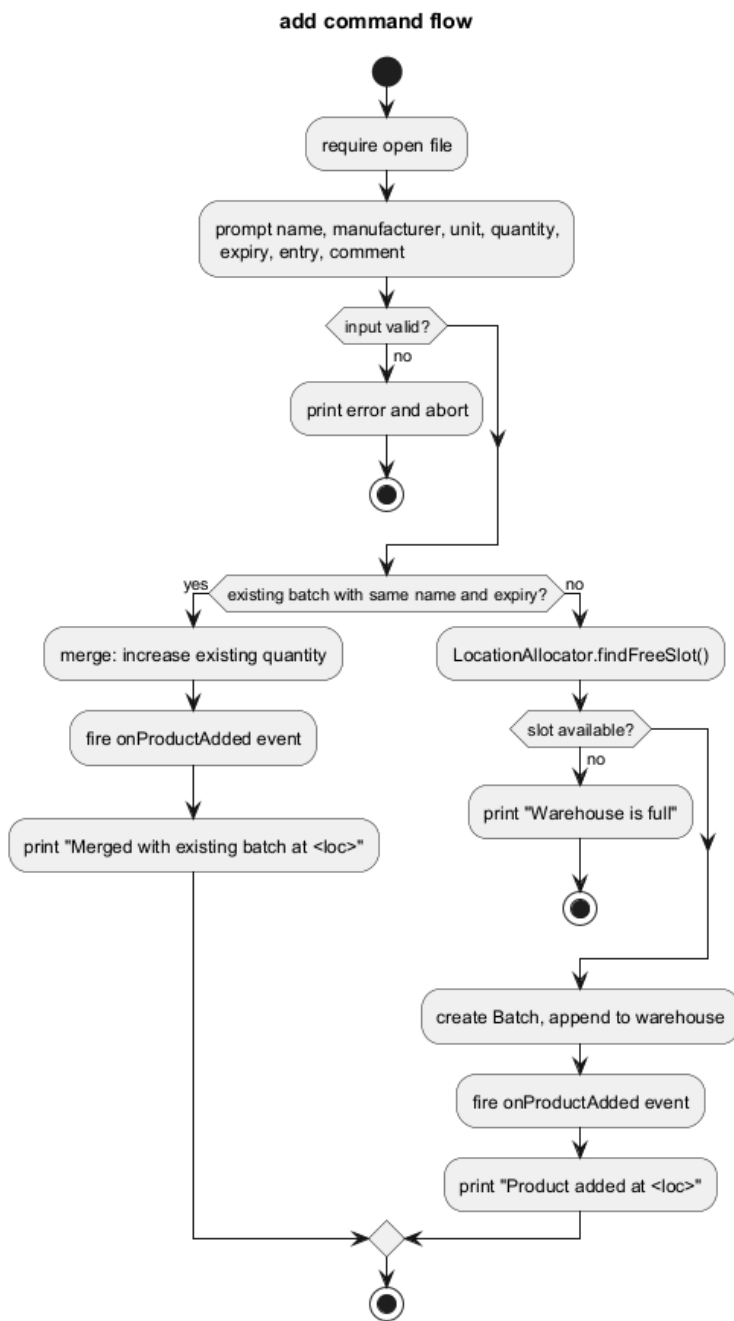
Фигура 5. REPL изпращане.

Конкретното изпълнение на `remove` — включително взаимодействието със стратегията и наблюдателя — на Фигура 6.

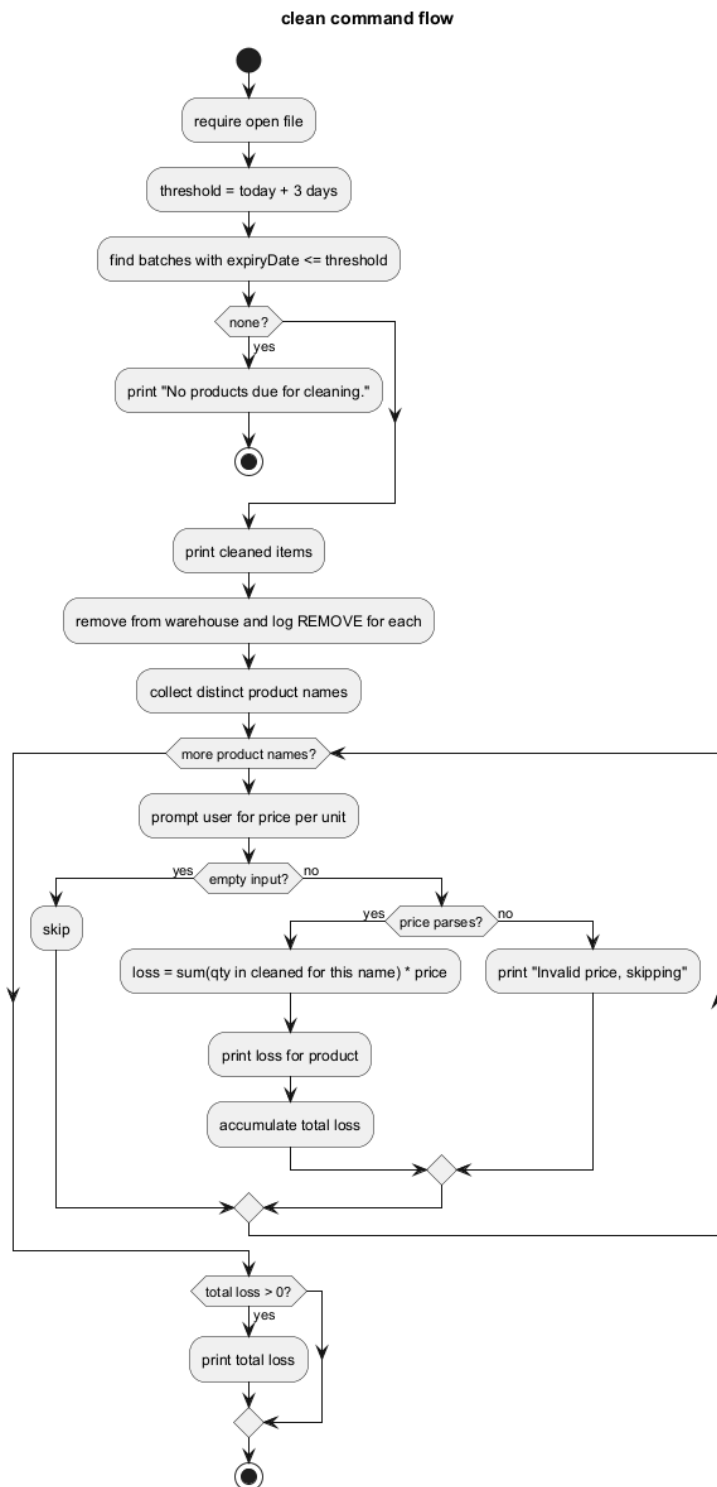


Фигура 6. *remove* командата.

Потоците на двете основни модифициращи команди — `add` и `clean` — са дадени като activity диаграми на Фигура 7 и Фигура 8.



Фигура 7. add — ноток.



Фигура 8. *clean* — ноток.

Глава 4. Реализация

4.1 Реализация на класове

REPL цикъл (CommandLineInterface). Главният цикъл отпечатва приканващ знак, чете ред, разделя на токени през `CommandFactory.tokenize`, намира съответната команда и я изпълнява. Особената команда `save as` се разпознава като двусловен ключ:

```
if ("save".equals(commandName) && tokens.length > 1
    && "as".equalsIgnoreCase(tokens[1])) {
    commandName = "save as";
}
```

Централизираната обработка на изключения от `Receiver` слоя:

```
try {
    command.execute(tokens);
} catch (NoFileOpenException | InvalidQuantityException
        | ProductNotFoundException e) {
    io.println(e.getMessage());
}
```

`WarehouseLoadException` не е в `catch` блока — тя е фатална и се пуска нагоре към `Main`, който отпечатва съобщението и приключва програмата с код 1 (заданието: „прекрати своето изпълнение“).

Receiver (WarehouseService). Централната точка за бизнес логиката. Командите делегират към него всички действия, които променят състоянието на склада. В конструктора получава стратегията за изваждане и държи списък от слушатели за `Observer` шаблона.

Команди (bg.warehouse.command.impl). Дванадесет конкретни класа, всеки реализира `Command.execute(String[])` и делегира бизнес логиката на `WarehouseService`. Командите не пишат директно `LogEntry` — това става през `AuditLogger`.

Стратегия (ExpiryFirstRemovalStrategy). Поддържа подадените кандидати по срок на годност възходящо и източва от началото. Подмяна с нова стратегия (LIFO, лотариен принцип) е една смяна на инстанция в `CommandFactory`:

```
RemovalStrategy strategy = new ExpiryFirstRemovalStrategy();
WarehouseService service = new WarehouseService(session, allocator, strategy);
```

Наблюдател (AuditLogger). Единственото място, което знае как се конструира LogEntry. Регистрира се във фабриката:

```
service.addListener(new AuditLogger(WarehouseSession.getInstance()));
```

XML слой (XmlFileHandler). Тънка обвивка над

JAXBContext.newInstance(Warehouse.class). LocalDateAdapter и LocalDateTimeAdapter пишат и четат датите във формат уууу-мм-дд и уууу-мм-ддТНН:мм:сс. Примерен запис на партида:

```
<batch>
  <productName>Flour</productName>
  <manufacturer>Mill Co</manufacturer>
  <unit>KILOGRAMS</unit>
  <quantity>50.0</quantity>
  <expiryDate>2027-06-01</expiryDate>
  <entryDate>2026-05-17</entryDate>
  <location>
    <section>A</section>
    <shelf>1</shelf>
    <slot>01</slot>
  </location>
  <comment>winter batch</comment>
</batch>
```

Изключения. Четири собствени изключения, всички наследници на RuntimeException:

- NoFileOpenException — хвърля се от WarehouseSession.requireOpen().
- InvalidQuantityException — от WarehouseService.drain(...) при не-положителна, NaN или Infinity стойност.
- ProductNotFoundException — от WarehouseService.requireBatchesByName(name).
- WarehouseLoadException — фатално: от OpenCommand при невалиден XML.

4.2 Алгоритми и оптимизации

Изваждане през стратегията. WarehouseService.drain валидира количеството, делегира на стратегията, премахва изпразнените партии и уведомява слушателите за всяко отделно изваждане:

```
public List<RemovalResult> drain(List<Batch> sorted, String name, double qty) {
    if (qty <= 0 || Double.isNaN(qty) || Double.isInfinite(qty)) {
```

```

        throw new InvalidQuantityException("Quantity must be positive.");
    }
    List<RemovalResult> results = removalStrategy.remove(sorted, name, qty);
    warehouse().getBatches().removeIf(b -> b.getQuantity() == 0);
    for (RemovalResult r : results) {
        fireRemoved(name, r.amountTaken(), r.batch().getLocation());
    }
    return results;
}

```

Историческа справка за загуба. `WarehouseService.lossInPeriod` чете журналните записи на склада и сумира количеството по записите REMOVE за дадения продукт в зададения период:

```

public double lossInPeriod(String productName, LocalDate from, LocalDate to) {
    double total = 0;
    for (LogEntry e : warehouse().getLogEntries()) {
        LocalDate d = e.getTimestamp().toLocalDate();
        if (d.isBefore(from) || d.isAfter(to)) continue;
        if (!"REMOVE".equals(e.getAction())) continue;
        if (!e.getProductName().equalsIgnoreCase(productName)) continue;
        total += e.getQuantity();
    }
    return total;
}

```

Сравнението е case-insensitive — съвпада с останалите методи на сервиза, които също игнорират разликата в регистъра при имена.

Праг за разчистване. Намират се всички партии със срок до три дни от днешна дата:

```

LocalDate threshold = LocalDate.now().plusDays(Constants.EXPIRY_WARNING_DAYS);
List<Batch> expired = service.findExpiringBy(threshold);

```

Константата `Constants.EXPIRY_WARNING_DAYS = 3` се пази на едно място, така че бъдеща промяна на политиката иска само една редакция.

Период за журнала. `queryLog` използва `!isBefore && !isAfter` за включващо сравнение по двата края на интервала, така че запис в 23:59:59 на крайната дата също попада в резултата.

Глава 5. Тестване

За защитата са подготвени седем тестови сценария (Таблица 4), които покриват както нормалните пътища, така и крайните случаи. Сценариите са под `demo/scripts/` и могат да се пуснат поотделно или общо чрез `demo/run-demo.ps1`.

Таблица 4. Тестови сценарии за защитата.

#	Сценарий	Покрива
1	<code>01-happy-path.txt</code>	Отваряне на нов файл, добавяне на два продукта, запис, затваряне, повторно отваряне, проверка
2	<code>02-fifo-drain.txt</code>	Три партии от едноименен продукт с различни срокове; <code>remove</code> източва най-скоро изтичащата първа
3	<code>03-not-enough.txt</code>	Изискване за изваждане на повече от наличното — отговори „не“ (отказ) и „да“ (взимане на остатък)
4	<code>04-log-loss.txt</code>	Журнал в период, историческа справка за загуба, проверка на <code>case-insensitive</code> сравнение
5	<code>05-clean.txt</code>	Файл с изтекъл, скоро-изтичащ и безопасен продукт; почистване, въвеждане на цени, общо изчисление на загубата
6	<code>06-malformed.txt</code>	Повреден XML — <code>open</code> хвърля <code>WarehouseLoadException</code> , програмата завършва с код 1

#	Сценарий	Покрива
7	07-save-as.txt	save as с кавичен път, преминаване към новия файл, повторно отваряне

Сценарий 2 (FIFO). След три добавки на хляб с срокове 2026-12-01, 2026-06-15 и 2026-05-25 и команда `remove Bread 25`:

```
Removing from batch [A-1-03, expiry: 2026-05-25]: 10.00 KILOGRAMS
Removing from batch [A-1-02, expiry: 2026-06-15]: 15.00 KILOGRAMS
Successfully removed 25.00 KILOGRAMS of Bread.
```

Скоро-изтичащата партида (A-1-03) е източена напълно; останалите 15 единици са взети от следващата по срок (A-1-02).

Сценарий 4 (loss). След добавяне на 40 литра масло и две изваждания (10 и 5 литра), `loss Oil 3.50 2026-05-01 2026-05-31` дава:

```
Lost 15.00 of Oil @ 3.50/unit = 52.50 over [2026-05-01..2026-05-31]
```

Същата команда с `OIL` (главни букви) връща идентичен резултат.

Сценарий 6 (повреден XML). Стандартният изход съдържа:

```
Error opening file: null
```

И процесът завършва с код 1.

Глава 6. Заключение

Поставените в Глава 1 цели са изпълнени. Реализирана е работеща програма, която чете и записва XML файлове, поддържа всички задължителни и допълнителната команда `loss`, и налага бизнес правилата за партидно складиране, FIFO изваждане и периодично почистване.

От архитектурна гледна точка проектът демонстрира шест шаблона за проектиране — `Command`, `Factory`, `Builder`, `Singleton`, `Strategy` и `Observer` — както и принципа на инверсия на зависимостите през интерфейса `ConsoleIO`. Добавянето на нова команда (например `loss`) изисква нов клас и един ред в `CommandFactory` — без промени в съществуващ код. Същото важи за нова стратегия за изваждане.

Крайните случаи са покрити: празен склад, изваждане на повече от наличното, изтекли продукти, повреден входен файл, празни или невалидни числови входове. Седем тестови сценария потвърждават всяко от ключовите изисквания.